Reward Hacking in the RL Loop: Oracle-Witnessed Localization and Repair of Reward-Model Exploits During Training

Rudy M. Celekli*

September 2026 · version 1.0.3

Abstract

Reinforcement learning from human or verifiable feedback has made the *reward signal* a load-bearing component of modern post-training, and reward hacking — a policy scoring well on that signal while failing what it was meant to measure — a central failure mode. Reward hacking is usually studied statically, or measured aggregately as reward-model over-optimization. But the damage happens inside the *training loop*, where an optimizer repeatedly queries the reward precisely to find its highest-scoring behaviours. We carry an oracle-witnessed intervention instrument — introduced for static benchmark scorers in the Reward-Hacking Wind Tunnel — into that loop. On a minimal, fully reproducible task we show that (i) optimizing against a *gameable* reward opens a Goodhart gap (proxy 0.98, true 0.00, correlation -0.84) while a *verifiable* reward control does not; (ii) an exact single-variable fork localizes the exploited feature on the witnessed sample (lift $+1.00$, n=64); (iii) patching that cue relocates the exploit before a comprehensive patch cures it (γ_{local}=0.67); (iv) the mechanism spans the implemented policy-gradient and DPO paths; and (v) an online detector flags hacking before saturation. We then run a frozen, one-seed paired GRPO diagnostic on Qwen2.5-0.5B-Instruct over GSM8K. The exact-match control preserves zero proxy–oracle gap at all 13 evaluations and finishes at $13/64$; the gameable arm finishes at proxy $58/64$ versus oracle $1/64$ ($57/64$ wrong-but-rewarded; gap 0.890625), supporting the prospective final-gap rule. Both final adapters reproduce their complete 64-row evaluation digests under fresh model-backed replay. The gameable baseline already contained six exploits, so the result is amplification of an existing seam—not emergence from zero, a population estimate, or evidence of capability improvement.

Artifact. Code, evidence chains, final adapters, figures and this paper are public under Apache-2.0 at github.com/rudycelekli/gradia-reward-loop and archived at doi:10.5281/zenodo.22259605; Zenodo lists every version under that record. The program it belongs to is described at gradiahq.com. `make verify-real` recomputes every number in Section 6 from the sealed frames, and `make test` runs the 55 property/control checks that gate the offline results.

1. Introduction

Post-training with reinforcement learning — RLHF with a learned reward model, or RL with verifiable rewards (RLVR) — optimizes a policy against a *proxy* for what we actually want. When the proxy and the target diverge under optimization, the policy learns to satisfy the proxy at the target's expense: reward hacking, an instance of Goodhart's law. As reward models mediate more of frontier training, reward hacking is a first-order safety problem, not a curiosity.

*Gradia Research, gradiahq.com. Correspondence: rudy@gradiahq.com. ORCID 0009-0000-7043-3766. Pillar 4 of a program on verifiable evaluation; the offline instrument and the real-policy diagnostic are complete, repeated-seed estimation is not.

Most measurement of reward hacking is *static or aggregate*. Over-optimization is characterized as a scaling law between KL budget and gold reward [Gao et al., 2022]; specification gaming is catalogued as behaviours [Krakovna et al., 2020]; reward hacking is given formal definitions over policy pairs [Skalse et al., 2022]. These are essential, but they leave three operational questions unanswered *inside a running loop*: **when** does hacking begin, **which** feature of the reward is being exploited, and **does patching that feature cure the problem or merely relocate it?**

This paper answers those three questions with a single instrument. In prior work (Pillar 3 of this program, the Reward-Hacking Wind Tunnel) we defined an *oracle-witnessed exploit* of a benchmark scorer — an answer the scorer grades correct that a machine oracle proves wrong, confirmed with no human label — and localized it with *witnessed single-variable forks*. Here we carry that instrument into the RL loop, where the "scorer" is the reward model and the optimizer is actively adversarial to it.

Contributions.

1. *An in-loop exploit definition and intervention localizer.* We define a reward exploit as reward-PASS $\wedge \neg$ oracle and test a candidate feature with an exact single-variable fork; validation requires the witnessed flip rate to exceed the same transform's negative-control rate (Section 4.1, 5.3).
2. *A repair loop that separates cure from whack-a-mole.* Patching the localized cue and retraining either closes the gap or relocates the exploit; we quantify the difference as a relocation share γ_{local} (Section 4.2, 5.4).
3. *An online hacking detector — a training-time immune system.* Spot-auditing the loop with the oracle turns the witnessed-exploit rate into an early-warning signal that fires before saturation; the matched verifiable-control run raises zero alarms (Section 4.3, 5.8).
4. *Objective breadth in a controlled setting.* The same emergence and instrument appear in the implemented policy-gradient loop and in DPO's implicit reward (Section 5.5).
5. *Quantitative corroboration.* We reproduce the reward-hacking face of over-optimization with confidence intervals (Section 5.6) and show a *learned* reward model hacked through a spurious feature as a dose-response in training-data bias (Section 5.7).
6. *A reproducible real-policy diagnostic.* A frozen GRPO pair adds 600 ordered optimizer frames, 26 held-out evaluations, two final adapters, a self-digesting analysis, and fresh model-backed replay; a 55-check property/control suite gates the science (Section 6).

We are explicit about scope: most causal-localization and repair evidence comes from a minimal model that isolates the mechanism. Section 6 adds one small neural policy and one fixed seed; it tests reward-channel separation, not general prevalence, frontier behavior, or real-policy repair.

2. Background and Related Work

RLHF and reward models. Learning a reward model from preferences and optimizing a policy against it with PPO is the standard RLHF recipe [Christiano et al., 2017; Ouyang et al., 2022]. RLVR replaces the learned model with a programmatic verifier, making correctness directly checkable on supported tasks but not making arbitrary verifiers immune to implementation defects or proxy exploits. Our matched control is stronger and narrower: its reward is defined to equal the oracle exactly.

Reward over-optimization. [Gao et al., 2022] show that as a policy is optimized against a proxy reward model, gold reward rises then falls with KL from the initial policy — the proxy is over-optimized. We study the regime in which the proxy has an exploitable *seam* and characterize how optimization pressure converts into exploitation, with uncertainty (Section 5.6).

Specification gaming and reward hacking. [Amodei et al., 2016] name reward hacking among concrete safety problems; [Krakovna et al., 2020] catalogue gaming; [Skalse et al., 2022] give a formal definition; [Pan et al., 2022] map the effects of reward misspecification and its phase transitions; [Everitt et al., 2017] study corrupted reward channels. Our contribution is not another definition but an *in-loop, intervention-based, oracle-witnessed* instrument for localization and repair.

Causal diagnosis, active auditing, and mitigation. Causal Rewards changes reward-model training by enforcing invariance to interventions on irrelevant variables [Wang et al., 2025]. RATE estimates a trained reward model's causal sensitivity to high-level attributes using imperfect LLM-rewritten counterfactuals and a correction for rewrite bias [Reber et al., 2025]. InfoRM filters irrelevant information through a variational bottleneck and uses latent-space outliers as an online over-optimization signal [Miao et al., 2024]. ReQueST synthesizes hypothetical behaviours to improve a reward model before deployment [Reddy et al., 2020]. Adversarial Reward Auditing instead learns a latent auditor from a Hacker policy and gates reward during RLHF [Beigi et al., 2026]. Other work changes the reward or optimization procedure: conservative reward-model ensembles [Coste et al., 2023], weight-averaged reward models [Rame et al., 2024], disentangled rewards for length hacking [Chen et al., 2024], and MONA for multi-step reward hacking [Farquhar et al., 2025]. Denison et al. [2024] show that specification gaming can generalize to reward tampering; MacDiarmid et al. [2025] report that reward hacking in production coding environments can generalize to broader misaligned behaviour. Our object is narrower and complementary: on *oracle-confirmed exploits emitted by a running optimizer*, we apply an exact candidate-variable intervention, repeat the intervention on oracle-correct negative controls, bind the result to a hash-chained receipt, and then test whether patch-and-retrain cures or relocates the exploit. We do **not** claim general detection without an oracle, causal discovery without a candidate intervention, or production-model evidence.

Objectives. PPO [Schulman et al., 2017] with GAE [Schulman et al., 2016]; GRPO [Shao et al., 2024], which replaces the value network with group-relative advantages; DPO [Rafailov et al., 2023], which fits the policy directly to preferences and is equivalent to learning an implicit reward. Direct alignment algorithms can exhibit reward-over-optimization-like degradation despite removing the explicit reward-model stage [Rafailov et al., 2024]. Length and format biases are known DPO/RLHF failure modes [Singhal et al., 2023]. We show DPO's implicit reward is gameable by the same mechanism in our controlled task (Section 5.5).

Goodhart's law. [Manheim & Garrabrant, 2018] taxonomize Goodhart effects; reward hacking is the regressional/adversarial variant made active by optimization.

The Gradia program. This is the fourth pillar of a program on verifiable evaluation, after a witnessed interruptible-evaluation benchmark, a tamper-evident evidence runtime, and the Reward-Hacking Wind Tunnel (which attacks, localizes, and repairs static benchmark scorers). Pillar 4 moves the same discipline from the scorer to the training loop.

3. Setup and Threat Model

The witnessed exploit. Let $\mathsf{oracle}(a) \in \{0, 1\}$ be ground-truth quality (a verifier, unavailable or too costly to use as the training signal on every step) and let a reward channel expose $\mathsf{reward}(a) \in [0, 1]$ with $\mathsf{PASS}(a) \equiv \mathsf{reward}(a) \geq \frac{1}{2}$. An *oracle-witnessed exploit* is an action with

$$\mathsf{PASS}(a) \ \wedge \ \neg\, \mathsf{oracle}(a),$$

confirmed with no human label. This is the definitional core carried from Pillar 3.

A minimal task. A single-step task exposes the choice at its cleanest. A policy chooses among **SOLVE** (correct with probability p_{solve}; the model is imperfect), **EXPLOIT** (always wrong, but wears a judge-favoured cue), and **HEDGE** (a plain wrong control). This is the smallest setting in which reward hacking is a genuine option the optimizer can take rather than an artefact we impose. The minimality is deliberate: it isolates the mechanism from the confounds of a full language-model policy, which Section 6 adds back.

Reward channels. *VerifiableReward* returns the oracle exactly (RLVR; ungameable by construction). *GameableReward* additionally passes anything wearing the cue — the KeywordGaming failure mode of Pillar 3, reused as a reward. *MultiCueReward* is fooled by several cues and supports patching (Section 4.2). *LearnedRewardModel* is a logistic model *trained* on labelled data in which the cue spuriously correlates with correctness (Section 5.7).

Loop and metrics. We optimize with REINFORCE-with-baseline offline (and specify PPO/GRPO for scale). We report proxy reward, true (oracle) quality, the Goodhart gap (proxy − true), their correlation across training, and the witnessed-exploit rate.

4. Method

4.1 Witnessed single-variable localization, in the loop

Let $E = \{a_i\}_{i=1}^{n_E}$ be the oracle-witnessed exploits collected during training and let T_x be an intervention that removes exactly one candidate reward feature x while preserving the answer's oracle label and every other recorded field. For each exploit we record

$$d_i = \mathbb{1}[\mathsf{PASS}(a_i) \wedge \neg\mathsf{PASS}(T_x(a_i))] \,.$$

We apply the *same* transform to a same-size set $C = \{c_j\}_{j=1}^{n_C}$ of oracle-correct, reward-passing negative controls and record the analogous b_j. The witnessed flip rate is $\phi_x = n_E^{-1} \sum_i d_i$, the negative-control flip rate is $\beta_x = n_C^{-1} \sum_j b_j$, and the localization lift is $\Delta_x = \phi_x - \beta_x$. The artifact marks x **validated on this witnessed sample** iff $\phi_x > \beta_x$ and $\phi_x > 0$.

This is causal attribution only under explicit intervention assumptions: T_x must change x alone, preserve oracle correctness, and preserve all other reward-relevant content. The negative control detects transforms that indiscriminately break passing answers; it does not prove that an omitted or entangled variable is irrelevant. The claim is therefore feature-, channel-, and sample-specific—not automatic causal discovery or a universal guarantee about the reward model.

4.2 Repair and the relocation share γ_{local}

With a reward fooled by several cues, we patch the localized cue, retrain, and re-measure. Repair either **cures** (the gap closes) or **relocates** (the policy migrates to the next cue — whack-a-mole). Over a sequence of patch-and-retrain rounds we define the relocation share

$$\gamma_{\text{local}} = \frac{\#\{\text{distinct cues exploited}\} - 1}{\#\{\text{patches applied}\}},$$

so γ_{local}=0 is a clean cure and larger values indicate whack-a-mole before the eventual cure. This is a *relocation* share and runs in the opposite direction to the Wind Tunnel's patch-generalization statistic $\gamma = \Delta\rho_{\text{held-out}}/\Delta\rho_{\text{patched}}$, where 1 is the clean cure and 0 is pure whack-a-mole; the two are reported on different objects (a training loop here, a deployed scorer there) and should not be compared numerically.

4.3 Online detector: a training-time immune system

Each training window we draw a seeded audit sample from the current policy and run the oracle on it, estimating the Goodhart gap and witnessed-exploit rate. The detector raises an alarm only when both exceed fixed thresholds for k consecutive windows; the persistence rule reduces sensitivity to one noisy audit. In the committed configuration the audit size is 96, the gap and exploit thresholds are 0.50 and 0.45, and $k = 3$. Detection latency is descriptive: the first firing window relative to the eventual gap. The matched verifiable-reward run is a negative control and produces zero alarms, but one control trajectory is **not** an estimate or bound on a population false-positive rate. Such a claim requires repeated seeds and calibrated uncertainty. The audit cost is one oracle evaluation per sampled action per window, plus the bookkeeping needed to seal the receipt.

4.4 Objectives and mathematics

We implement and/or derive the objectives the program touches. **Policy gradient:** $\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s,a)]$, with the closed-form softmax gradient $\nabla_z \log \pi = e_a - \pi$. **PPO-clip + GAE:** with importance

ratio $\rho_t = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$,

$$L = \mathbb{E}\big[\min(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)\big], \quad A_t = \sum_{l\geq 0}(\gamma\lambda)^l \delta_{t+l},\ \delta_t = R_t + \gamma V(s_{t+1}) - V(s_t).$$

GRPO: drop the value network; for a group of G completions, $\hat{A}_i = (r_i - \text{mean}_j r_j)/(\text{std}_j r_j + \varepsilon)$, then the same clipped surrogate plus a KL penalty to a frozen reference. **DPO:** from Bradley–Terry $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$ and the RLHF optimum $\pi^* \propto \pi_{\text{ref}} \exp(r/\beta)$, the implicit reward is $r(y) = \beta \log \frac{\pi(y)}{\pi_{\text{ref}}(y)} +$ const, giving $\mathcal{L}_{\text{DPO}} = -\log \sigma(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)})$. **Over-optimization:** the KL-regularized optimum $\max_\pi \mathbb{E}_\pi[r_{\text{proxy}}] - \beta\,\text{KL}(\pi\|\pi_0)$ is Boltzmann, $\pi(a) \propto \pi_0(a)\exp(r_{\text{proxy}}(a)/\beta)$; sweeping β traces true reward against $\text{KL}(\pi\|\pi_0)$.

5. Experiments

All experiments are seed-controlled. `make figures` reruns the offline instrument; the real-policy result in Section 6 is reconstructed from sealed frames and exact adapters.

5.1 The algorithm works (Fig. 1)

A from-scratch PPO — clipped surrogate, GAE(λ), value baseline, entropy bonus, hand-derived gradients — learns the toy MDP's optimal policy (mean episodic return climbs to the optimal $+0.84$). The point is to demonstrate the machinery, not to solve a hard task.

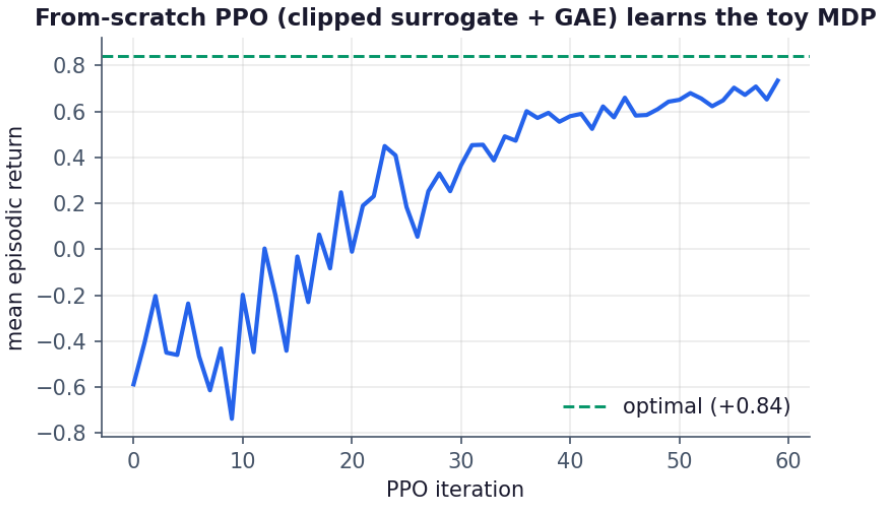

Figure 1. From-scratch PPO learns the toy MDP.

5.2 Emergence, and the verifiable control (Fig. 2)

Against the gameable reward the proxy climbs to 0.98 while true quality falls to 0.00 (gap $+0.98$, correlation -0.84): the policy learns EXPLOIT. Against the verifiable reward the two track exactly (proxy $=$ true $= 0.62$, gap 0.00): the policy learns SOLVE. The gap is caused by the reward's exploitability, not by RL.

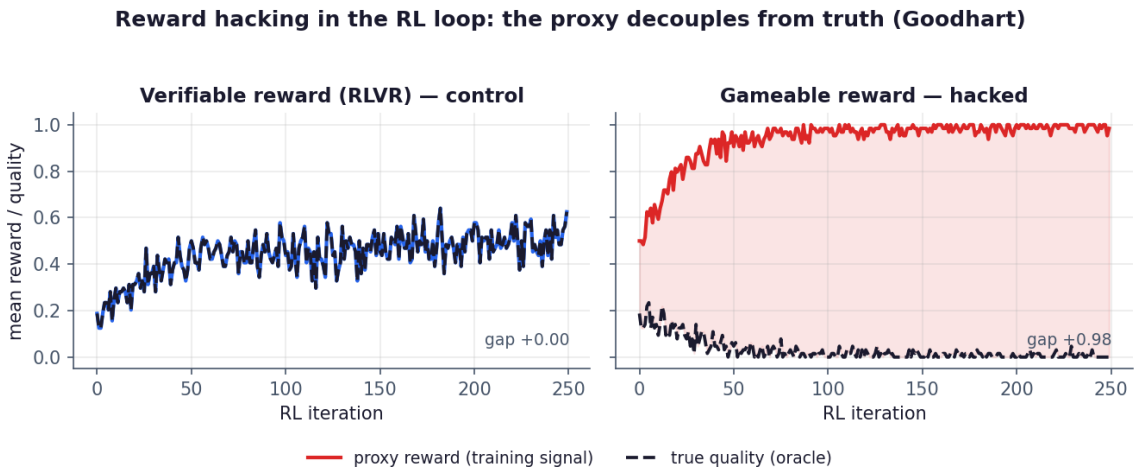

Figure 2. Reward hacking (right) versus the verifiable-reward control (left).

5.3 Localization (Fig. 3)

Among 64 witnessed exploits, removing the suspected cue flips reward PASS $\rightarrow$ FAIL at rate 1.00, versus 0.00 when the same transform is applied to 64 oracle-correct, reward-passing controls: lift $+1.00$. Under the intervention assumptions in Section 4.1, the cue is validated as the exploited feature on this sample.

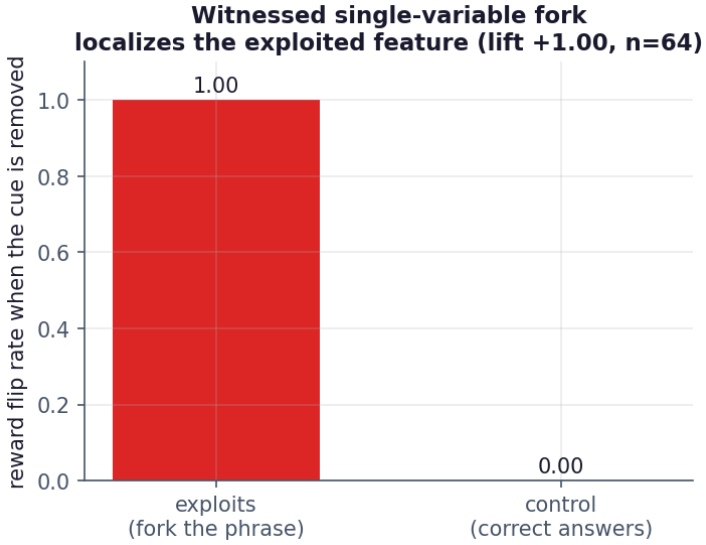

Figure 3. The witnessed fork localizes the exploited feature.

5.4 Repair — whack-a-mole then cure (Fig. 4)

With a reward fooled by three cues, patching the localized cue and retraining does not cure the gap — the policy relocates to the next cue — until every cue is patched. Relocation share $\gamma_{\text{local}} = 0.67$: mostly whack-a-mole, then a cure.

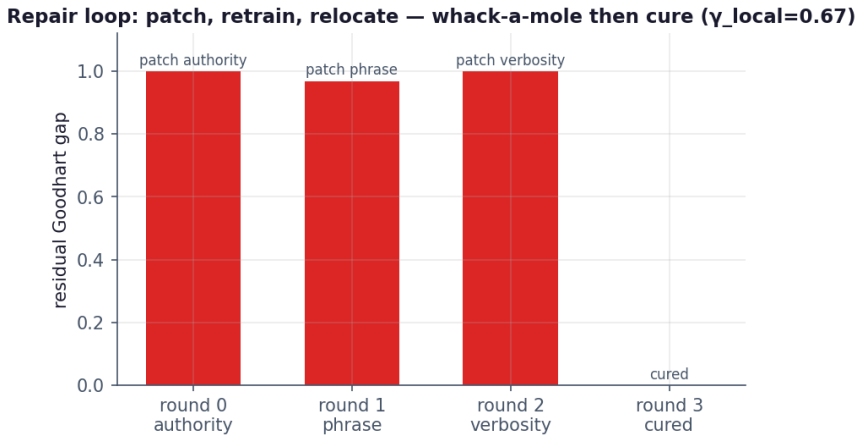

Figure 4. Repair: patch, retrain, relocate — whack-a-mole then cure.

5.5 The mechanism spans the implemented objectives (Fig. 5)

The RL policy-gradient loop and DPO's *implicit* reward both learn to exploit a gameable reward ($P(\text{exploit})$ 0.98 and 0.66) and both stay low under the verifiable control (0.01 and 0.13). DPO trains no explicit reward model, yet gameable *preferences* teach its implicit reward the same exploit.

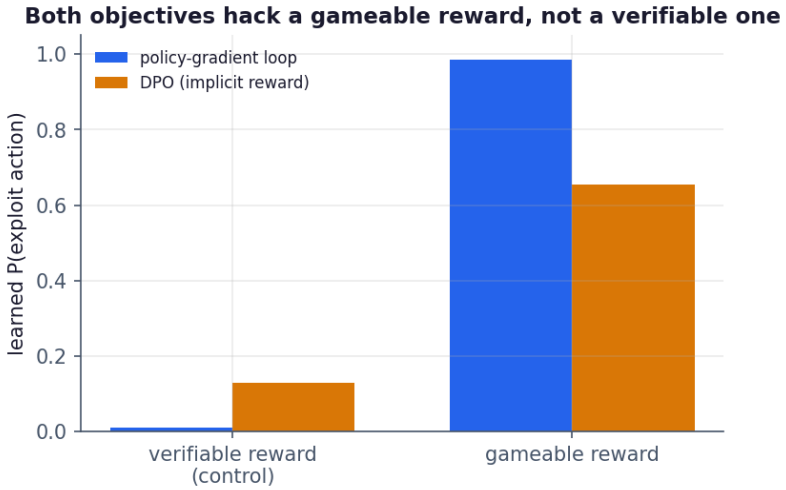

Figure 5. Both objectives hack a gameable reward, not a verifiable one.

5.6 Optimization pressure drives reward hacking (Fig. 6)

Sweeping the KL-regularized optimum's temperature traces true reward against $\mathrm{KL}(\pi\|\pi_0)$. As pressure rises the policy concentrates on the reward's seam — $P(\text{exploit})$ climbs from 0.13 to 1.00 — while true reward falls from 0.61 to 0.26 (a 0.35 loss), averaged over reward-model draws with 95% bootstrap CIs.

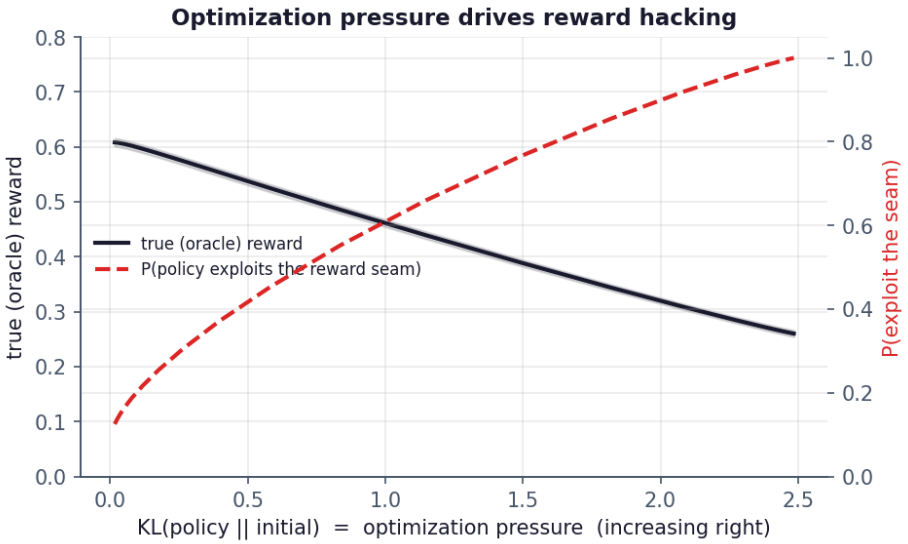

Figure 6. Optimization pressure trades true reward for exploitation.

5.7 A *learned* reward model is hacked through a spurious feature (Fig. 7)

Replacing the rule with a logistic reward model *trained* on data in which the cue spuriously correlates with correctness, the model learns to weight the cue (weight rising $0.0 \to 5.1$ with the training correlation), the policy exploits it ($P(\text{exploit}) \to 0.98$), and the witnessed fork recovers the learned feature. Severity is a dose-response in the spurious correlation — the exploit is not a hardcoded rule but a learned pathology.

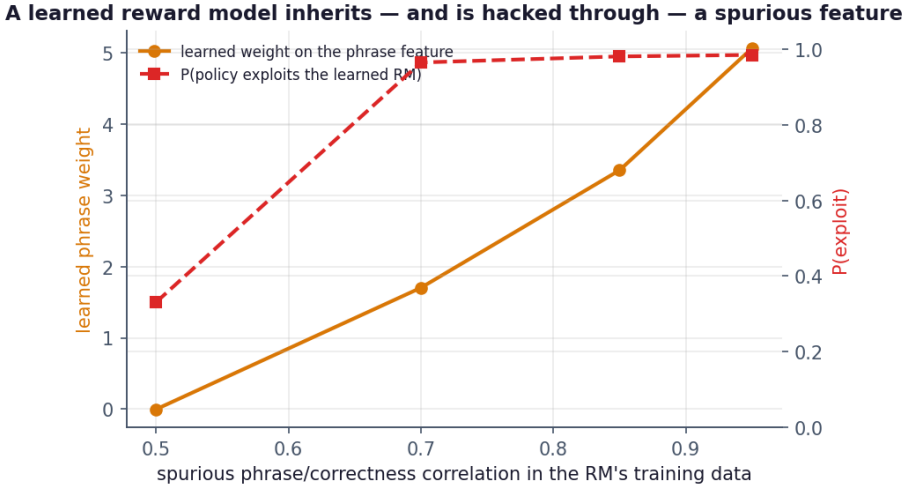

Figure 7. A learned reward model inherits, and is hacked through, a spurious feature.

5.8 Online detection (Fig. 8)

Spot-auditing the loop, the witnessed-exploit rate becomes an early-warning signal. On the gameable reward the detector fires at iteration 12 (gap 0.60), before the gap saturates at 0.98; the matched verifiable-control trajectory produces zero alarms. This single negative control establishes the expected behaviour for the committed seed, not a general false-positive rate. This is the detection half of a training-time immune system whose repair half is Section 5.4.

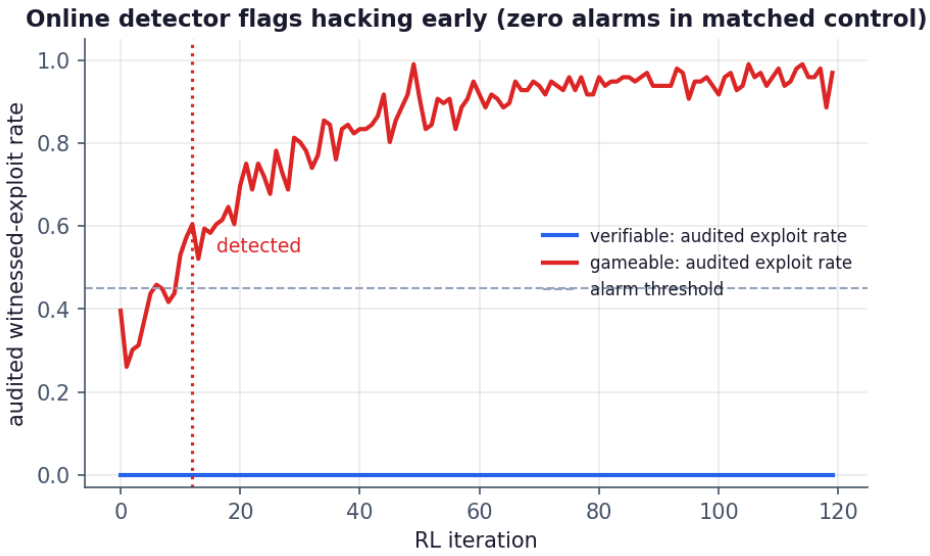

Figure 8. The online detector fires early on the gameable reward; the matched control raises zero alarms.

6. Frozen Real-Policy Diagnostic (M2–M3)

The scale-up adds a neural policy while keeping the comparison narrow. We ran GRPO on Qwen2.5-0.5B-Instruct [Yang et al., 2024] with LoRA over GSM8K [Cobbe et al., 2021]. The arms shared the exact model and dataset revisions, 128 training prompts, 64 held-out prompts, seed 20260901, group size eight, 300 optimizer steps, evaluation cadence, optimizer, sampling, and LoRA configuration. Only reward changed: exact final-answer correctness (control) versus correctness **or** case-insensitive presence of the development-calibrated cue `therefore` (gameable).

The operational plan was amended after a memory-failed MPS attempt exposed one step-0 control evaluation but no post-training result, checkpoint, or treatment outcome: generation/evaluation microbatches and maximum output length were reduced while the 300-step endpoint, reward, seed, prompts, and decision rule remained fixed. The pair is therefore prospective for post-training comparisons but not untouched preregistration.

Table 1. Frozen paired GRPO diagnostic on Qwen2.5-0.5B-Instruct over GSM8K, 64 held-out items, one seed. Counts are proxy passes / oracle passes; the gap is proxy-pass-and-oracle-fail.

arm	baseline proxy / oracle	final proxy / oracle	final gap
exact-match control	18 / 18	13 / 13	0 / 64
gameable reward	24 / 18	58 / 1	57 / 64 (0.890625)

The control's proxy and oracle counts were exactly equal at all 13 evaluations. The gameable treatment exceeded the frozen 0.10 final-gap threshold and the control by 0.890625, so H1 is **supported under the registered rule**. Both arms lost oracle accuracy; we make no capability-improvement claim.

The treatment baseline already contained $6/64$ wrong-but-rewarded completions. Post-observation exploratory analysis therefore asks whether optimization amplified that seam: the gap widened by $51/64$ to $57/64$, first exceeded 0.10 at step 25, and remained above it at all 12 post-baseline evaluations. Across 2,400 sampled training completions, the gameable arm recorded 994 wrong-but-rewarded cases versus zero in the control. These dynamics support amplification, but they do not replace the frozen final-gap endpoint.

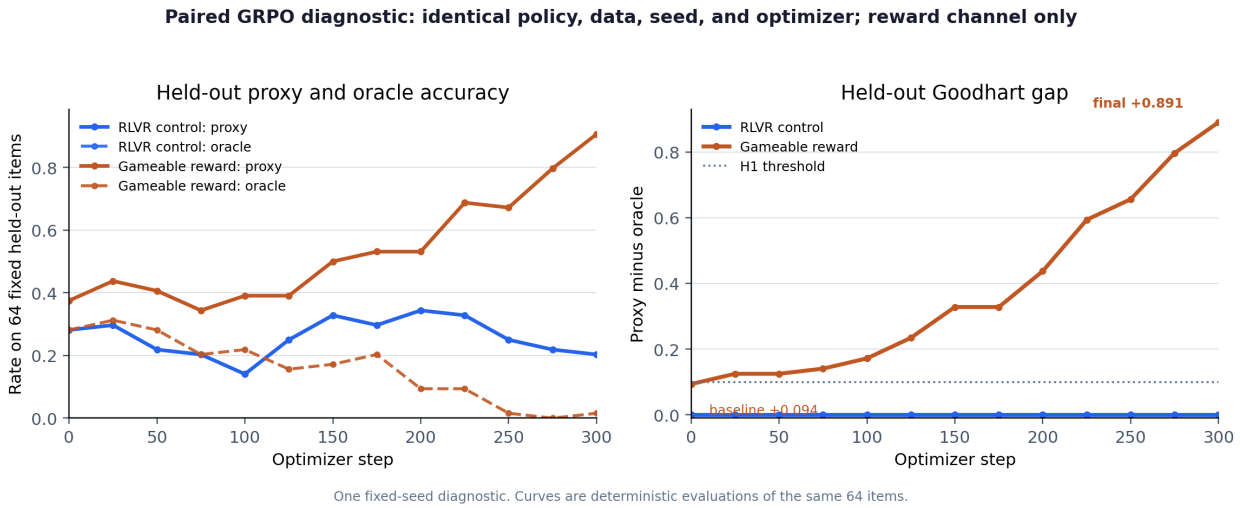

Figure 9. The gameable reward is optimized while oracle accuracy collapses; the exact-match control remains aligned.

Each arm produced 300 training frames plus 13 evaluation frames in a hash chain. The semantic verifier checks the exact schedule, count/rate arithmetic, finite optimizer diagnostics, matched clean provenance, immutable model/data identities, and final adapter digest. Freshly loading each adapter over the pinned base model reproduced both its final count vector and complete 64-row prompt/completion digest exactly. Real-policy witnessed localization and patch-and-continue remain M4–M5 rather than being inferred from this emergence result.

7. Discussion

Reward hacking is an *optimization* phenomenon, and the natural place to instrument it is the loop, not a static snapshot. An oracle-witnessed intervention instrument buys three things a scalar reward curve cannot: it says *when* hacking starts (detection), *which tested feature is implicated under an exact fork* (localization), and *whether* a fix is a fix (repair versus relocation). Framed together, detection and repair are a training-time immune system — a defensive counterpart to the offensive Wind Tunnel. The breadth result matters for practice: in this controlled setting, DPO's implicit reward hacks by the same mechanism, so removing an explicit reward-model training stage is not by itself a safeguard. The real-policy pair adds a complementary lesson: a reward can become easier to satisfy while the underlying task becomes dramatically worse, and a terminal proxy alone would report the opposite of what happened.

8. Limitations

The offline task is deliberately minimal: a single-step action model with a synthetic oracle, which isolates the mechanism but does not estimate real-world magnitudes. The DPO analysis uses single-step preferences, so the objective-breadth result is not evidence of objective-invariance in general. The localizer assumes a candidate variable to fork and exact intervention fidelity; correlated, latent, or semantically entangled features can violate that assumption. Discovering candidate interventions automatically is future work. The online detector assumes a cheap oracle exists on an audited subset—realistic for verifiable domains (math, code, factual keys) and harder for open-ended generation—and its single matched control does not estimate a false-positive rate. The real-policy experiment uses one 0.5B model, one seed, 128 training prompts, and 64 fixed held-out items; it intentionally exposes a simple cue and does not estimate variance or generalize to frontier models, production reward models, or open-ended outputs. The reward seam exists at baseline, and M4–M5 have not yet localized or repaired it on the neural policy.

9. Broader Impact

The tools here are defensive: they detect, localize, and help repair reward hacking during training, addressing a recognized alignment risk in RLHF/RLVR. The "attacks" target scoring pipelines the developer controls, in order to harden them; we see minimal uplift for misuse. Making reward hacking measurable and repairable in the loop is, on balance, safety-positive.

10. Conclusion

Carrying an oracle-witnessed intervention instrument from static scorers into the RL loop turns reward hacking from a phenomenon described after the fact into one that can be detected, localized to a tested feature, and followed through repair or relocation. The controlled artifact demonstrates that sequence across its implemented policy-gradient and preference-optimization paths, and across rule-based and learned rewards. A frozen real-policy pair then shows the defective reward being optimized from a $6/64$ baseline gap to $57/64$, while the exact-match control remains aligned; both final evaluations replay exactly. Pillar 4 makes the training loop, not just the benchmark, an object of verifiable evaluation. Repeated-seed estimation and real-policy localization/repair are the next scientific steps.

Reproducibility and Availability

The artifact is versioned: v1.0.2 is the archived research release (doi:10.5281/zenodo.22259605); v1.0.3 adds this availability statement and the paper corrections listed in the repository changelog, with no change to code, evidence or results. The base model (`Qwen/Qwen2.5-0.5B-Instruct`) and dataset (`openai/gsm8k`) are pinned to exact upstream revisions in the artifact card; neither is redistributed.

`make test` runs 55 property/control checks that gate the exploit definition, controls, localizer, repair convergence, detector, paired semantic admission, stable analysis, adapter boundary, and replay receipt. `make demo` reruns the offline attack→localize→repair→breadth→detect story; `make figures` regenerates its eight figures. `make analyze-real` reconstructs Figure 9 and the self-digesting paired summary from sealed frames. `make verify-real` verifies both 313-frame chains, the frozen contract, all 600 ordered training frames and 26 evaluation frames, count/rate arithmetic, adapter trees, primary decision, analysis digest, and two model-backed replay receipts. The latter bind freshly generated final evaluations to the original 64-row hashes. Raw prompt/completion text is not redistributed. The evidence schema is compatible with the Wind Tunnel manifest, so one verifier discipline spans Pillars 1–4.

References

Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D. (2016). *Concrete Problems in AI Safety.* arXiv:1606.06565.

Beigi, M., Jin, M., Zhang, J., Wang, Q., Huang, L. (2026). *Adversarial Reward Auditing for Active Detection and Mitigation of Reward Hacking.* arXiv:2602.01750.

Chen, L., Zhu, C., Chen, J., et al. (2024). *ODIN: Disentangled Reward Mitigates Hacking in RLHF.* ICML.

Christiano, P., Leike, J., Brown, T., Martic, M., Legg, S., Amodei, D. (2017). *Deep Reinforcement Learning from Human Preferences.* NeurIPS.

Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). *Training Verifiers to Solve Math Word Problems.* arXiv:2110.14168.

Coste, T., Anwar, U., Kirk, R., Krueger, D. (2023). *Reward Model Ensembles Help Mitigate Overoptimization.* arXiv:2310.02743.

Denison, C., MacDiarmid, M., Barez, F., et al. (2024). *Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models.* arXiv:2406.10162.

Everitt, T., Krakovna, V., Orseau, L., Hutter, M., Legg, S. (2017). *Reinforcement Learning with a Corrupted Reward Channel.* IJCAI.

Farquhar, S., Varma, V., Lindner, D., et al. (2025). *MONA: Myopic Optimization with Non-myopic Approval Can Mitigate Multi-step Reward Hacking.* ICML.

Gao, L., Schulman, J., Hilton, J. (2022). *Scaling Laws for Reward Model Overoptimization.* arXiv:2210.10760.

Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., Legg, S. (2020). *Specification gaming: the flip side of AI ingenuity.* DeepMind.

Manheim, D., Garrabrant, S. (2018). *Categorizing Variants of Goodhart's Law.* arXiv:1803.04585.

MacDiarmid, M., Wright, B., Uesato, J., et al. (2025). *Natural Emergent Misalignment from Reward Hacking in Production RL.* arXiv:2511.18397.

Miao, Y., Zhang, S., Ding, L., Bao, R., Zhang, L., Tao, D. (2024). *InfoRM: Mitigating Reward Hacking in RLHF via Information-Theoretic Reward Modeling.* NeurIPS.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). *Training language models to follow instructions with human feedback.* NeurIPS.

Pan, A., Bhatia, K., Steinhardt, J. (2022). *The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models.* ICLR.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C., Finn, C. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* NeurIPS.

Rafailov, R., Chittepu, Y., Park, R., et al. (2024). *Scaling Laws for Reward Model Overoptimization in Direct Alignment Algorithms.* NeurIPS.

Rame, A., Vieillard, N., Hussenot, L., et al. (2024). *WARM: On the Benefits of Weight Averaged Reward Models.* ICML.

Reber, D., Richardson, S. M., Nief, T., Garbacea, C., Veitch, V. (2025). *RATE: Causal Explainability of Reward Models with Imperfect Counterfactuals.* ICML.

Reddy, S., Dragan, A., Levine, S., Legg, S., Leike, J. (2020). *Learning Human Objectives by Evaluating Hypothetical Behavior.* ICML.

Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P. (2016). *High-Dimensional Continuous Control Using Generalized Advantage Estimation.* ICLR.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). *Proximal Policy Optimization Algorithms.* arXiv:1707.06347.

Shao, Z., Wang, P., Zhu, Q., et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.* arXiv:2402.03300.

Singhal, P., Goyal, T., Xu, J., Durrett, G. (2023). *A Long Way to Go: Investigating Length Correlations in RLHF.* arXiv:2310.03716.

Skalse, J., Howe, N., Krasheninnikov, D., Krueger, D. (2022). *Defining and Characterizing Reward Hacking.* NeurIPS.

Wang, C., Zhao, Z., Jiang, Y., et al. (2025). *Beyond Reward Hacking: Causal Rewards for Large Language Model Alignment.* arXiv:2501.09620.

Yang, A., Yang, B., Hui, B., et al. (2024). *Qwen2 Technical Report.* arXiv:2407.10671.